트랜스포머 처음부터 이해하기

1.1

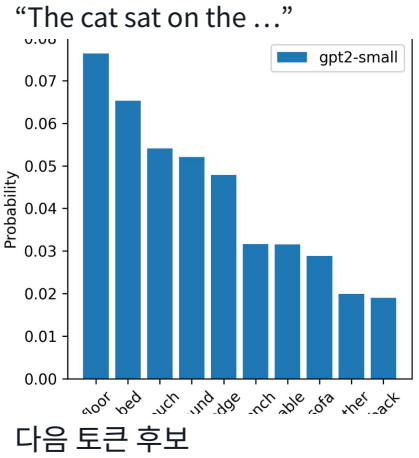
문장을 숫자로 바꾸고, 문맥을 계산하고, 다음에 올 말을 고릅니다.

ChatGPT는 어떻게 작동할까요?

- 오늘 살펴볼 구조는 트랜스포머입니다.
- 입력은 글을 나눈 작은 조각, 토큰의 나열입니다.
- 출력은 다음 토큰 후보 각각의 확률입니다.
- 예시 모델 GPT2 소형
  - 매개변수 약 1억 2,400만 개[1]
  - 학습에 사용한 텍스트 약 40GB

[1] 32비트 가중치만 약 496MB.
실행에는 추가 메모리가 필요합니다.

확률



다음 토큰 후보

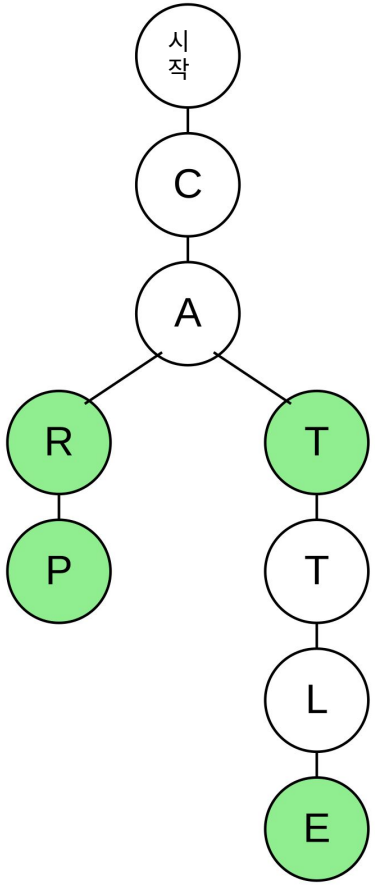
출력은 완성된 문장이 아니라, 다음 토큰 후보들의 확률입니다.



- 직접 문장을 넣어 토큰 나누기 를 살펴보세요.
- 각 토큰에는 ID가 있습니다.
- 모델은 이 번호들을 입력받습니다.

추가 영상 GPT 토크나이저 만들기

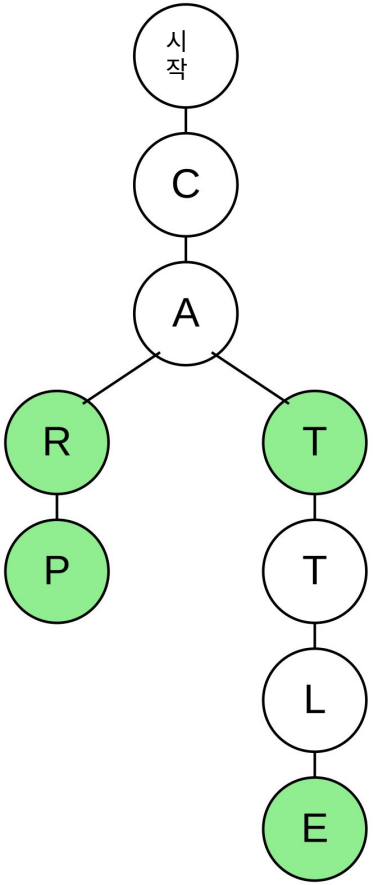
토큰은 글의 조각이고, 토큰 ID는 그 조각에 붙인 번호입니다.



토크나이저는 어떻게 작동할까요?

- 어휘표는 글 조각과 정수를 연결하는 표입니다.
  $\{cat, cattle, car, carp\} \rightarrow \{0, 1, 2, 3\}$
- 토크나이저는 먼저 만들고, 모델 학습 중에는 고정합니다.
- GPT2는 BPE 압축 방식을 활용합니다.

어휘표는 글 조각과 번호를 연결합니다. 의미를 학습하는 표와는 다릅니다.



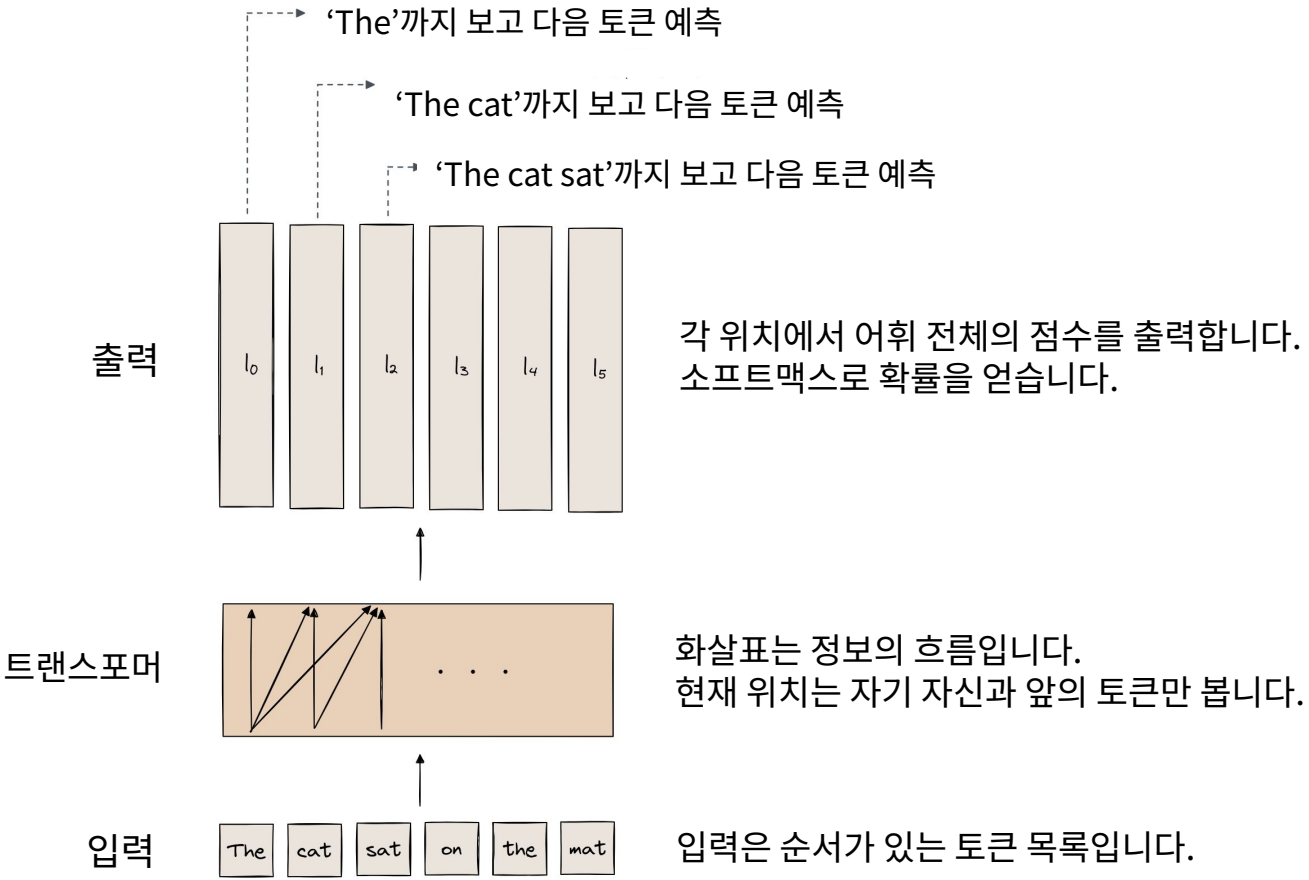
토크나이저를 조금 더 살펴보기

- 어휘표는 글 조각과 정수를 연결하는 표입니다.

$$\{cat, cattle, car, carp\} \rightarrow \{0, 1, 2, 3\}$$

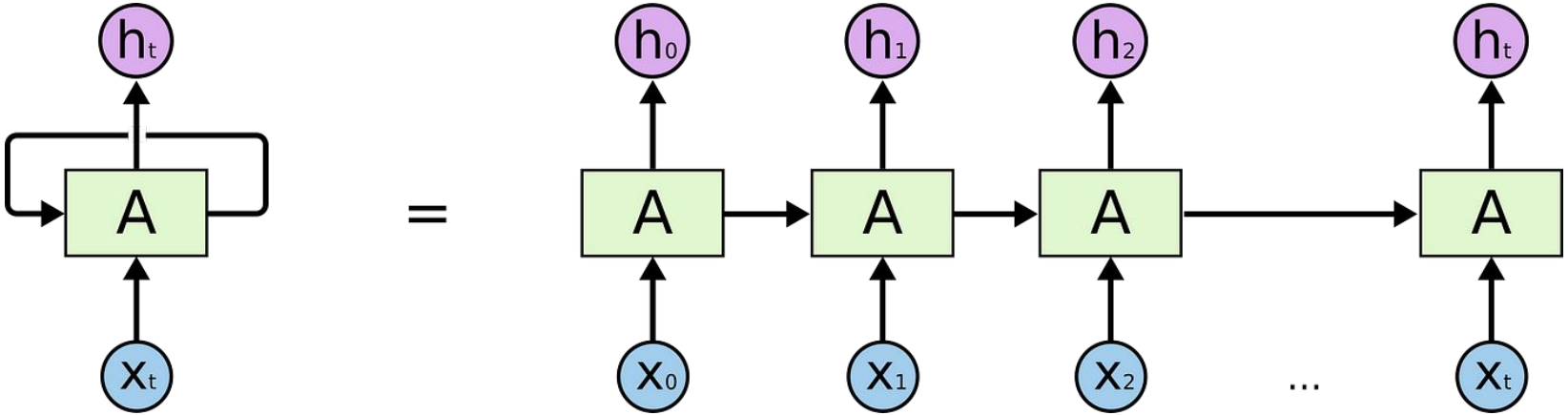
- 문자열 검색 자료구조인 트라이
- 문서 경계 등을 나타내는 특수 토큰도 있습니다.
- 그림은 문자열 검색의 예시입니다. GPT2의 BPE와는 구분합니다.
- 오늘은 준비된 GPT2 토크나이저를 사용합니다.
- 모델마다 어휘표와 토큰을 나누는 방식이 다를 수 있습니다.
- 토크나이저는 모델 학습 전에 만들고 고정합니다.
- GPT2는 BPE 압축 방식을 활용합니다.

오늘은 준비된 GPT2 토크나이저를 사용하고, 입력과 출력의 의미에 집중합니다.



학습할 때는 여러 위치의 다음 토큰을 함께 예측하되, 미래의 정답은 가립니다.

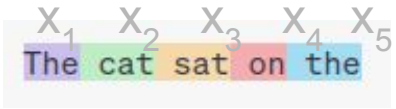
- RNN은 한 문장 안에서 앞 단계의 계산을 기다립니다.



RNN은 앞 단계 계산을 기다립니다. 트랜스포머는 학습 중 여러 위치를 함께 계산합니다.

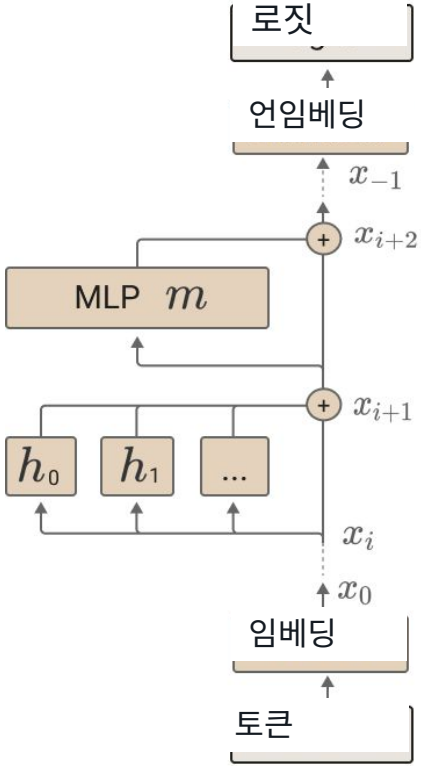
학습한 모델로 문장 만들기 **한 토큰씩 이어 쓰기**

- 입력 문장으로 시작합니다. “The cat sat on the”



- 토큰 ID들을 모델에 넣고,
  다음 토큰 후보들의 확률을 얻습니다.
- 어떤 토큰을 고를까요?
- 가장 확률이 높은 토큰 고르기 → 반복이 생길 수 있습니다.
- 확률에 따라 뽑기 → 확률이 높은 후보가 더 자주 선택됩니다.
- 온도 조절하기 → 높이면 후보 선택이 더 다양해질 수 있습니다.
- 고른 토큰을 입력 뒤에 붙입니다.
- 길어진 입력으로 다시 예측하고, 선택하는 과정을 반복합니다.

예측 → 하나 선택 → 입력 뒤에 붙이기. 이 과정을 반복하면 문장이 됩니다.



최종 벡터를 어휘별 점수로 바꿉니다.

$$T(t) = W_U x_{-1}$$

MLP가 계산한 결과를 기존 정보에 더합니다.

$$x_{i+2} = x_{i+1} + m(x_{i+1})$$

각 어텐션 헤드의 결과를 기존 정보에 더합니다.

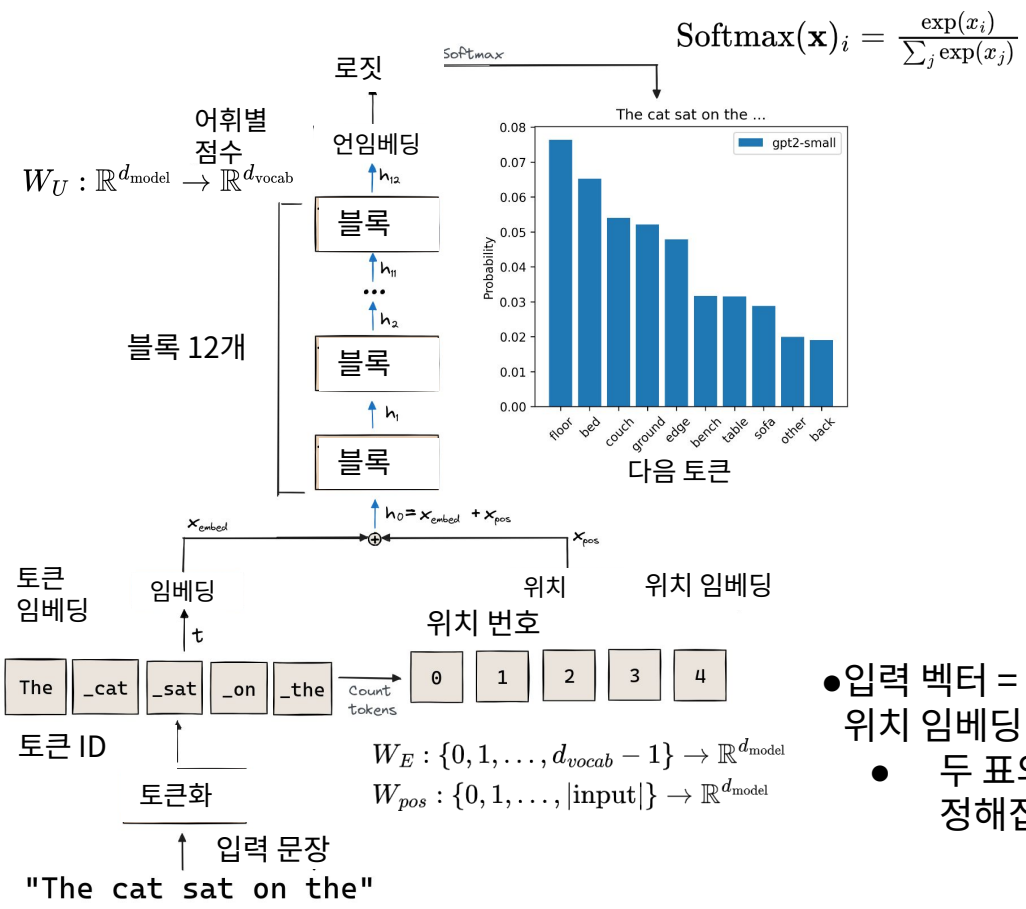
$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

토큰을 벡터로 바꿉니다.

$$x_0 = W_E t$$

잔차 블록 하나

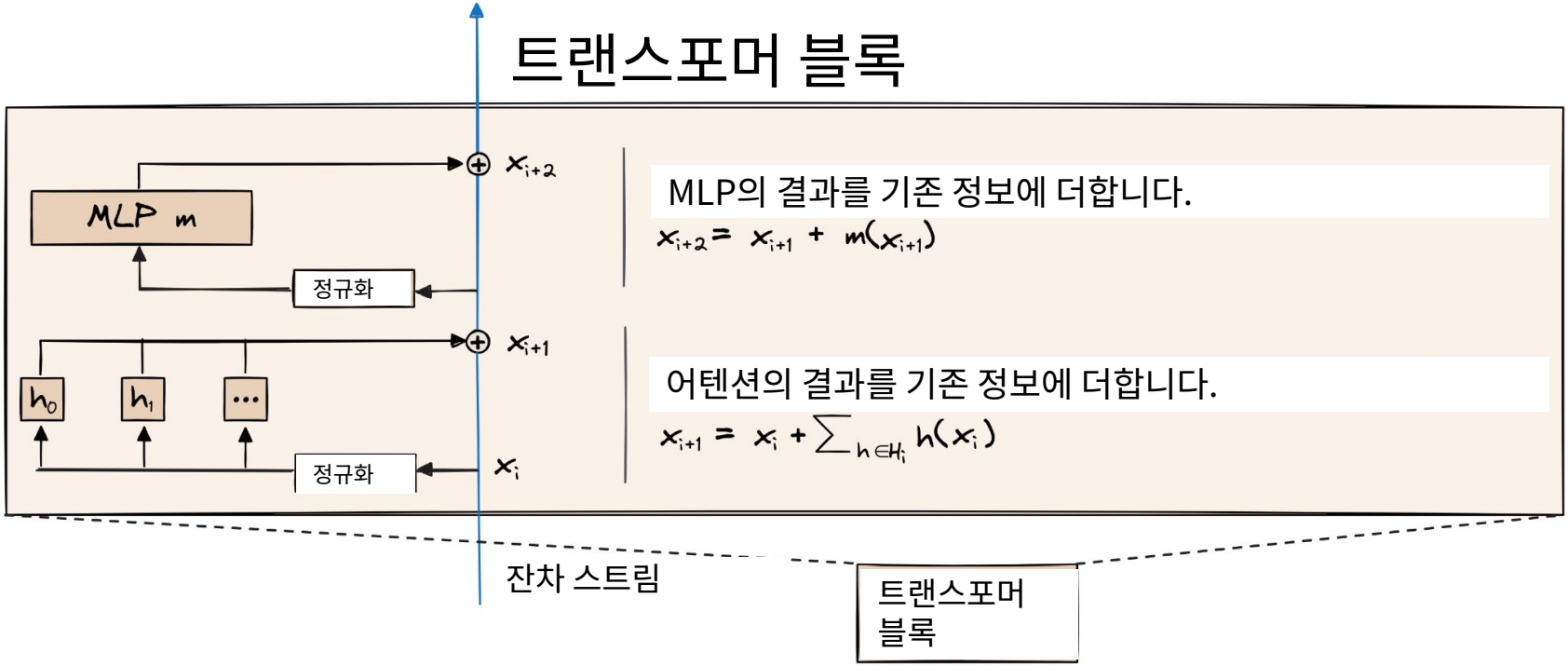
기존 정보를 유지하면서, 어텐션과 MLP가 계산한 내용을 차례로 더합니다.



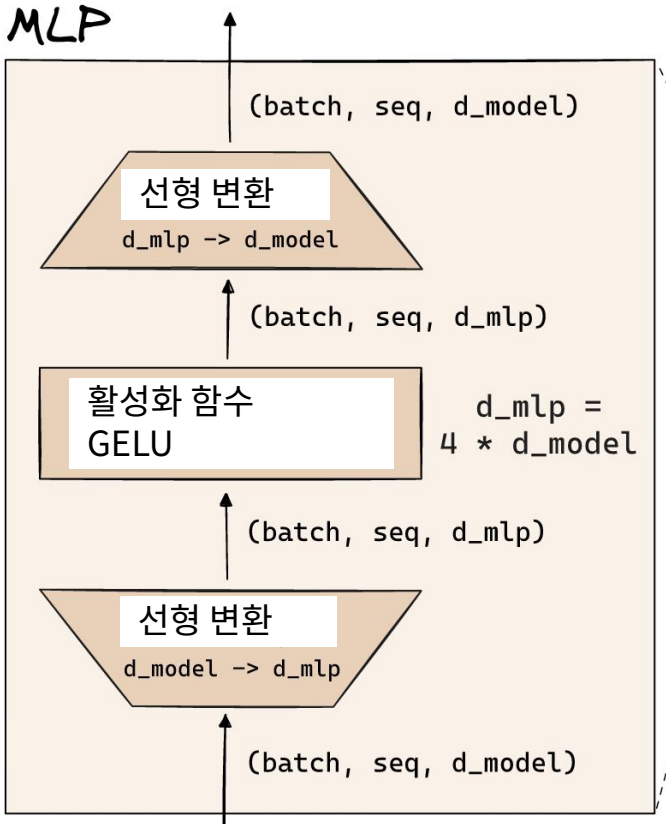
- 입력 벡터 = 토큰 임베딩 + 위치 임베딩
  - 두 표의 숫자는 *학습*으로 정해집니다.

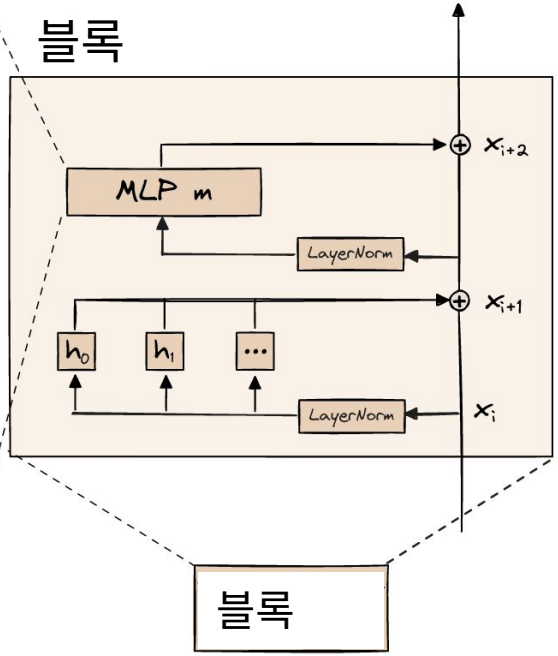
토큰 임베딩에 위치 임베딩을 더해, 무엇이 어디에 있는지 함께 표현합니다.

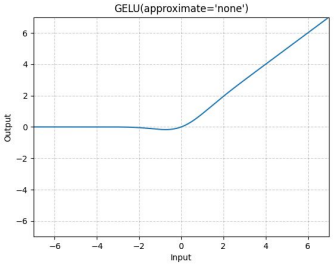
트랜스포머 블록



어텐션은 다른 위치의 정보를 모으고, MLP는 각 위치의 정보를 가공합니다.







$$\mathrm{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\frac{t^2}{2}}\, dt$$

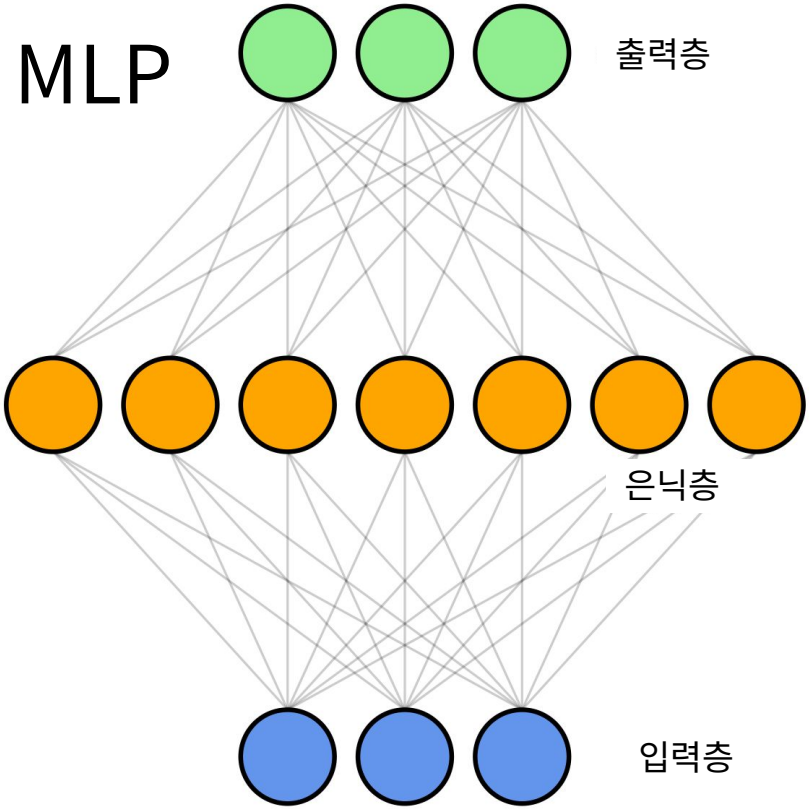
GELU는 비선형 변환입니다.
적분식은 외우지 않아도 됩니다.

숫자를 섞어 넓히고, 비선형 함수를 거친 뒤, 원래 길이로 되돌립니다.

벡터에는 여러 특징이 겹쳐 담길 수 있습니다

- N차원에서 서로 직교하는 벡터는 최대 N개입니다.
- 조금씩 겹치는 방향까지 허용하면 훨씬 많이 담을 수 있습니다.
- 여기서 기억할 점
  - 벡터의 한 칸이 개념 하나와 꼭 대응하지는 않습니다.
  - 차원이 커지면 표현의 여지가 늘지만, 성능 향상을 보장하지는 않습니다.
  - 여러 특징이 공간을 겹쳐 쓰는 현상을 ‘중첩’이라고 합니다.

여러 특징이 같은 벡터 공간을 겹쳐 사용할 수 있습니다. 이 장은 선택 심화입니다.



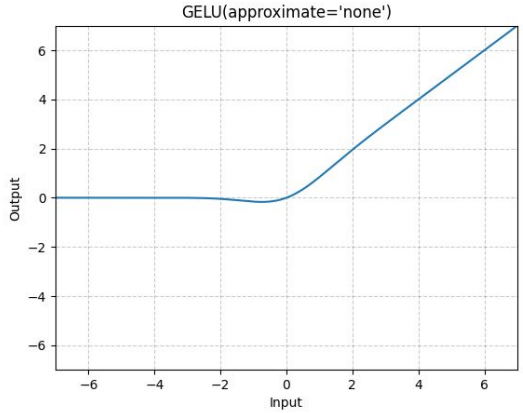
- 이 MLP는 은닉층 하나를 둔 신경망입니다.

$$\mathrm{MLP}(x) = W_2(\sigma(W_1 x + b_1)) + b_2$$

- σ 는 비선형 *활성화 함수입니다.*
- GPT2는 GELU 계열 함수를 사용합니다.

$$\mathrm{GeLU}(x) = x\Phi(x) \approx \max(x, 0)$$

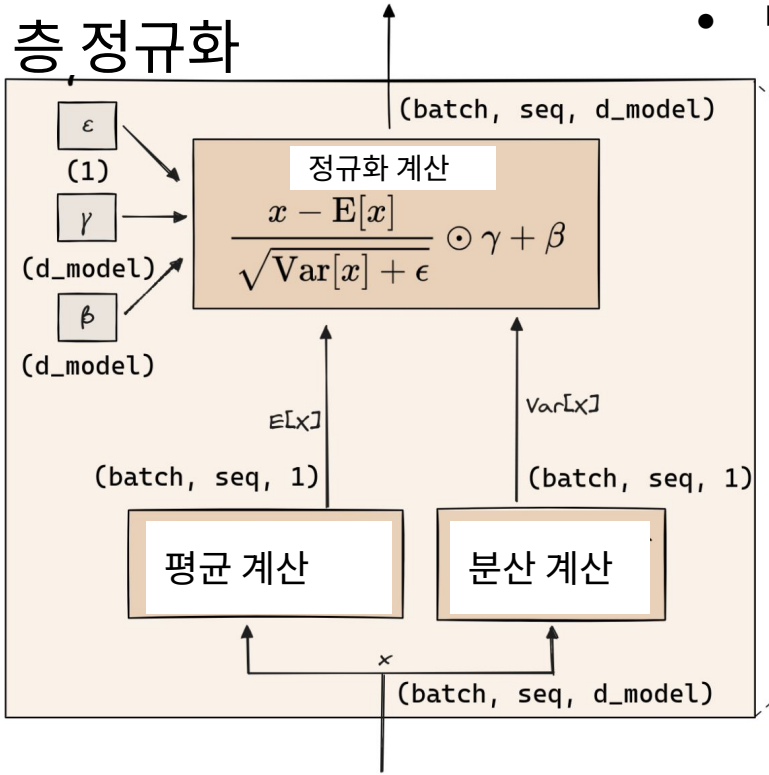
- $\Phi(x)$ 는 표준정규분포의 누적확률입니다. (참고)
- 다른 예로 ReLU, Tanh 등이 있습니다.



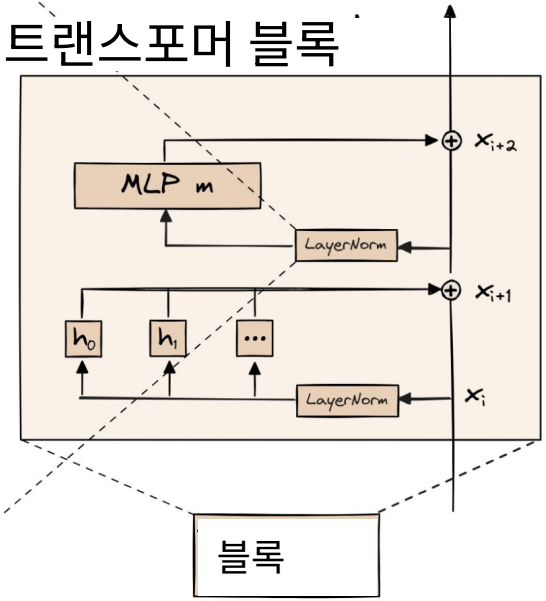
연결선의 가중치와 비선형 변환을 함께 사용해 입력 정보를 가공합니다.

- 토큰별 숫자 크기를 조절합니다.
- 마지막에 γ와 β를 적용합니다.

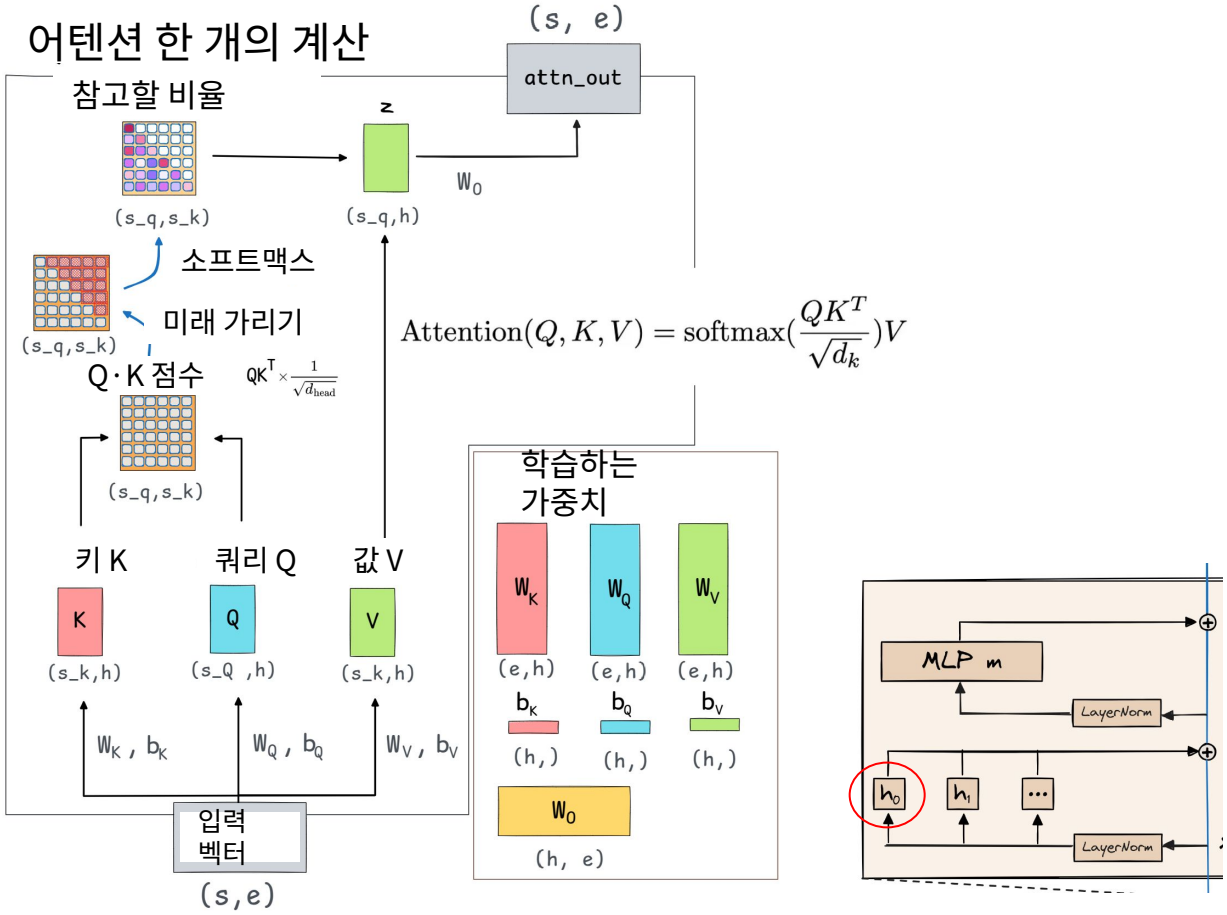
층 정규화



트랜스포머 블록



한 토큰의 숫자 묶음을 정규화한 뒤, 학습된 크기와 이동량을 적용합니다.



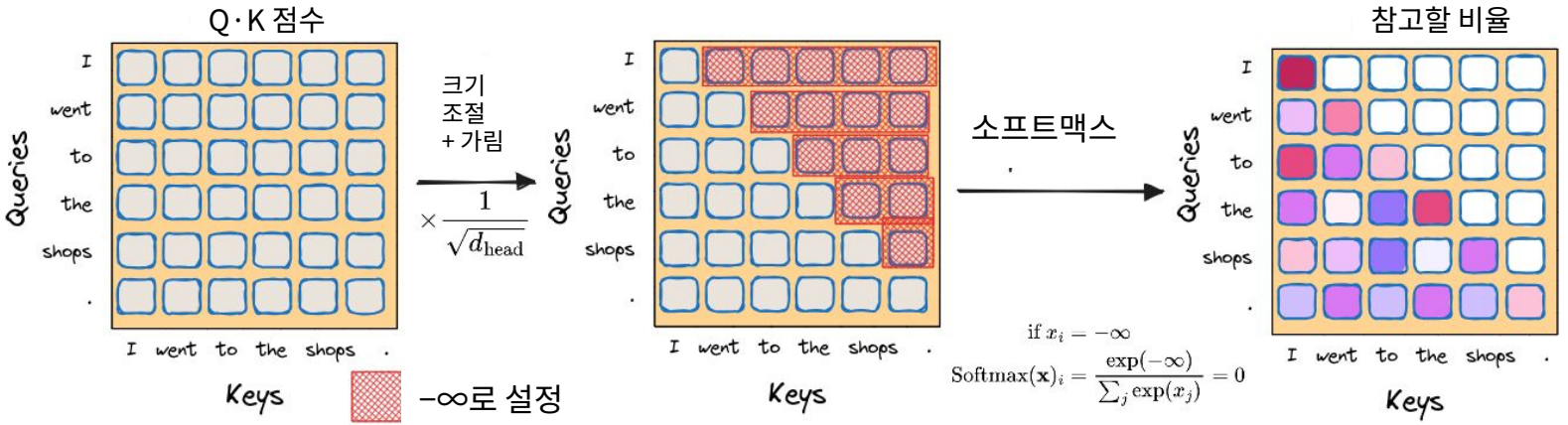
Q와 K로 참고할 비율을 정하고, 그 비율로 V를 섞습니다.

미래를 가리는 인과 마스크

- 현재 위치보다 뒤에 있는 키는 참고할 수 없게 막습니다.
  - 아직 주어지지 않은 정답을 미리 보지 못하게 합니다.

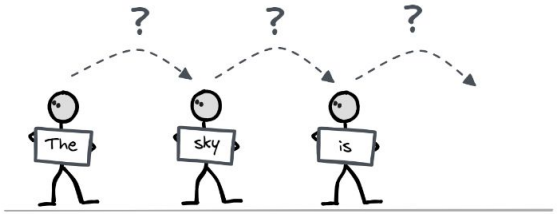
미래 위치는 보지 못하게 막습니다.

각 행의 비율 합은 1입니다.

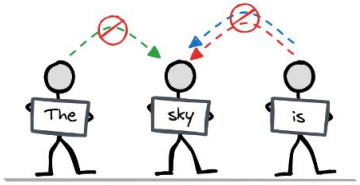


현재 위치는 자기 자신과 앞의 토큰만 참고할 수 있습니다.

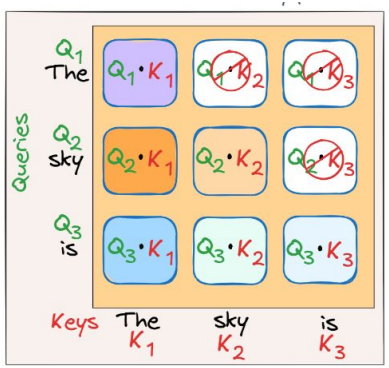
각 위치는 앞의 정보를 보고
다음 토큰을 예측합니다.



미래 위치는 가립니다.
자기 자신은 볼 수 있습니다.

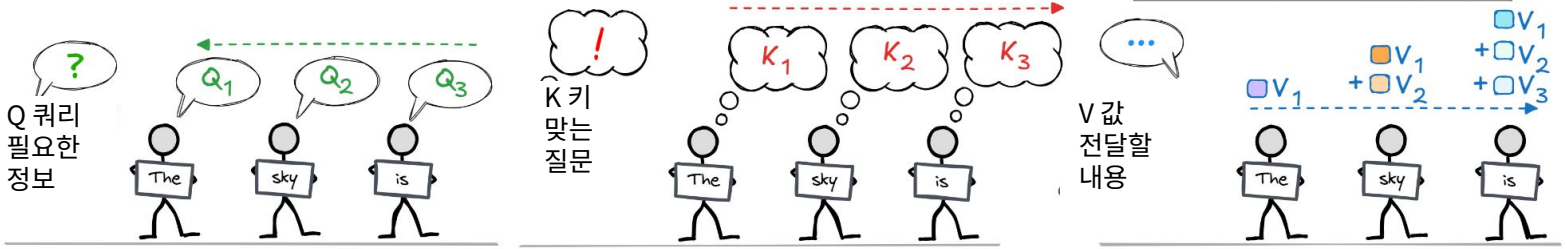


행은 질문 위치 Q
열은 참고할 위치 K



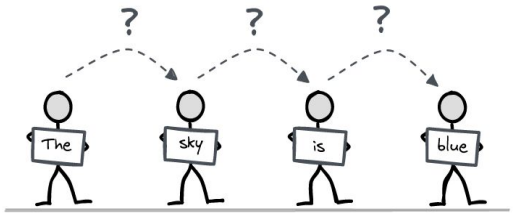
Q와 K로 참고 비율을 정하고,
그 비율로 V를 섞습니다.

자기 자신도 참고할 수
있습니다.

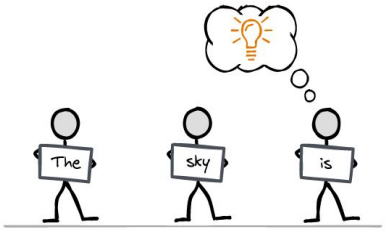


질문 Q와 설명표 K가 잘 맞는 곳에서, 전달할 내용 V를 가져옵니다.

앞의 토큰을 참고해
다음 토큰을 예측합니다.

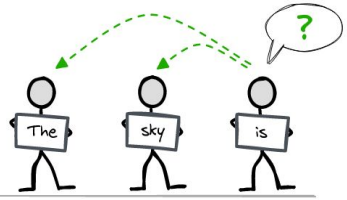


MLP는 각 위치에 모인 정보를
가공합니다. 다른 위치를
직접 가져오는 단계는 아닙니다.



어텐션은 앞의 위치에서
관련된 정보를 모읍니다.

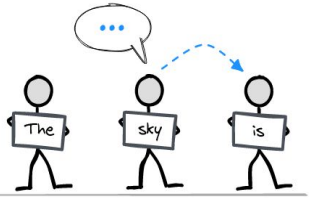
Q 쿼리
필요한
정보



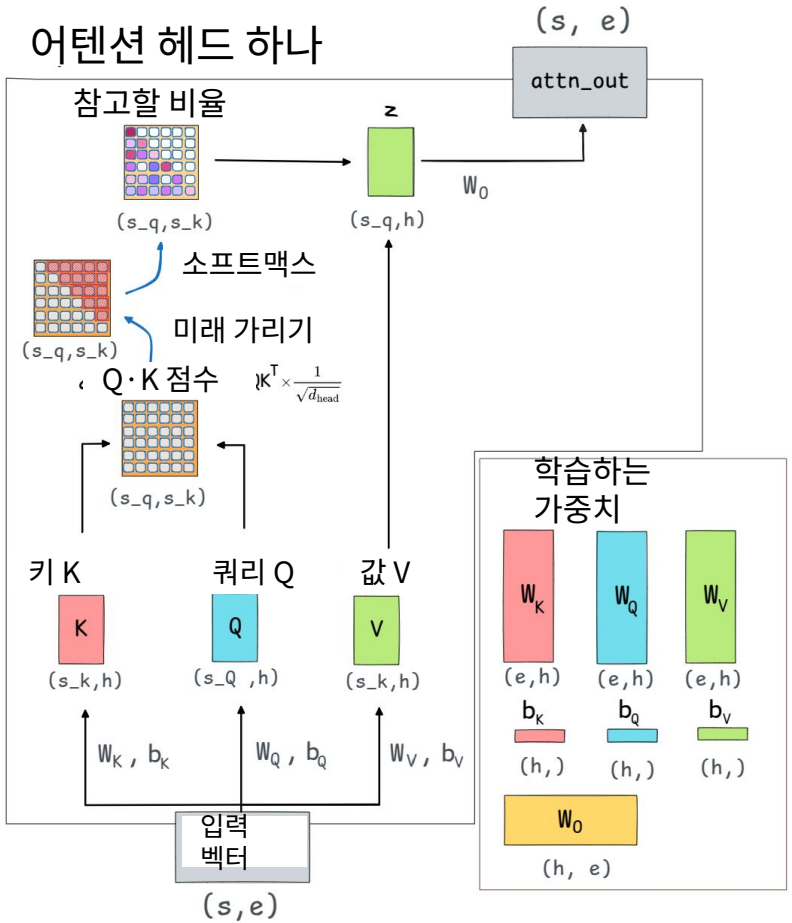
K 키
맞는
질문

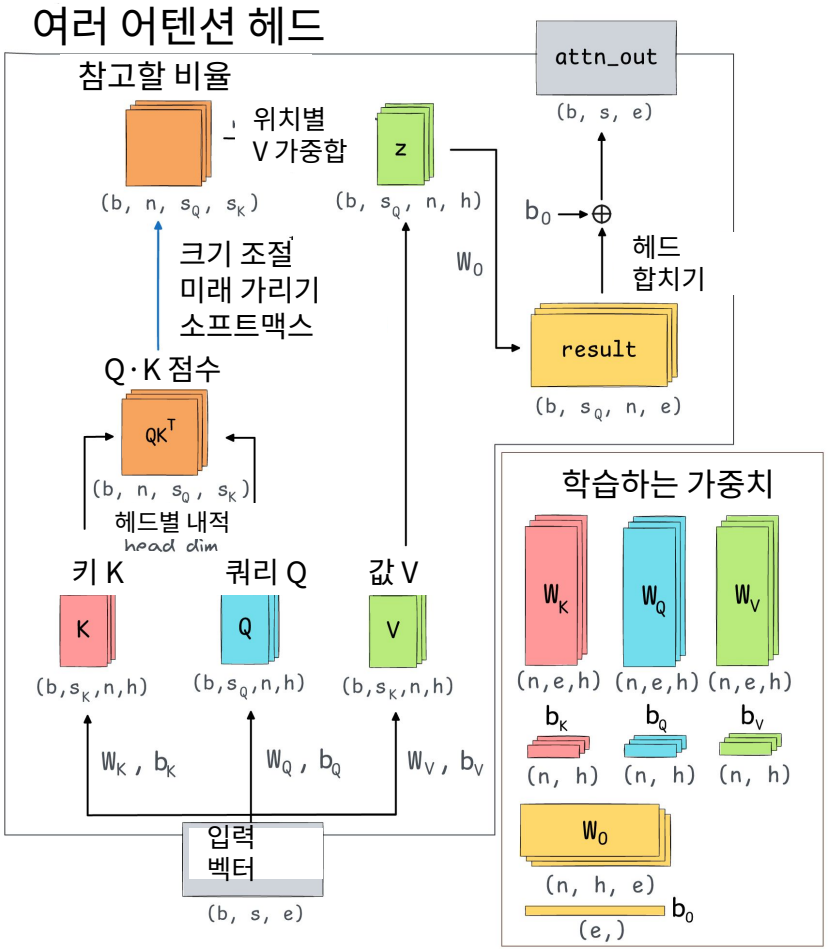


V 값
전달할
내용



어텐션으로 다른 위치의 정보를 모으고, MLP로 모인 정보를 처리합니다.





여러 헤드가 각자의 비율로 정보를 모으고, 결과를 합쳐 다음 단계에 전달합니다.

어텐션 헤드

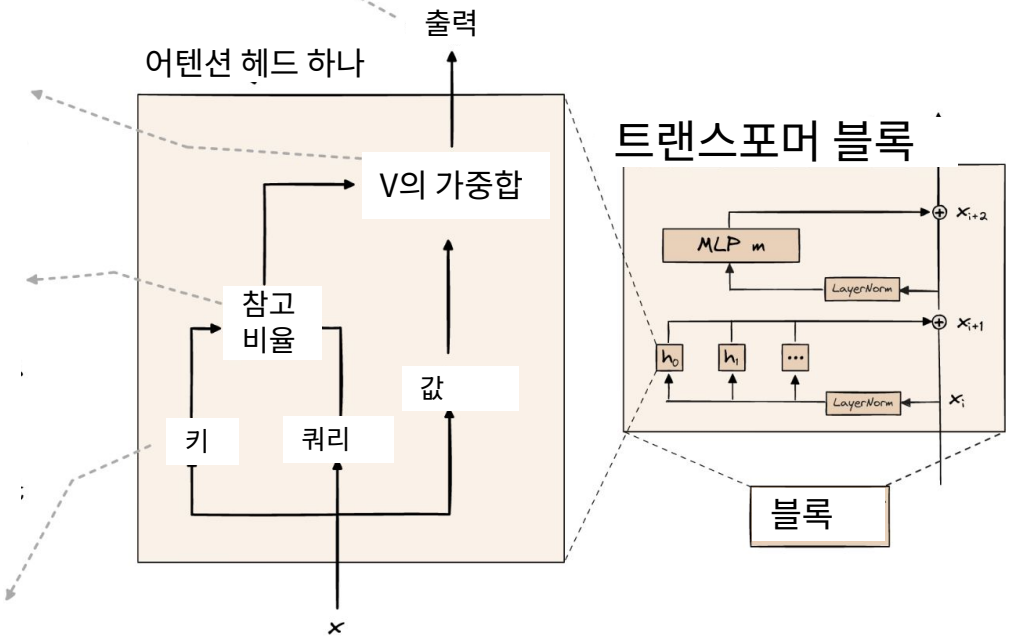
출력 변환으로
잔차 스트림에 더할 수 있는
벡터 길이로 바꿉니다.

각 위치에서 V를 가중합합니다.
참고 비율이 클수록
더 큰 계수로 들어옵니다.

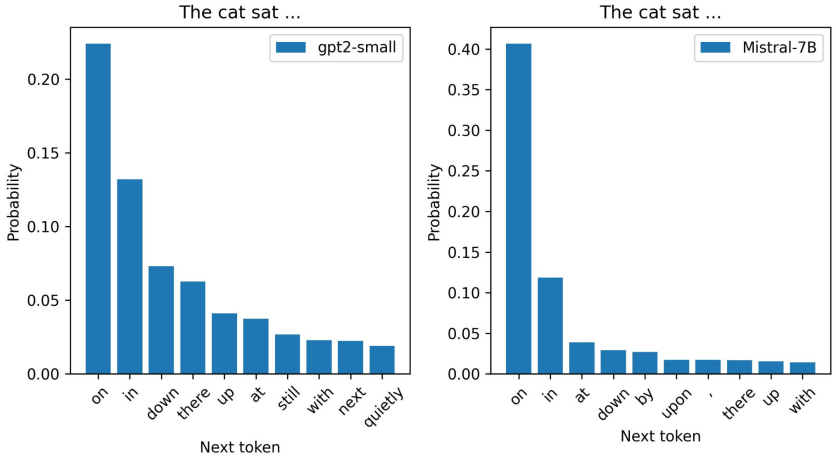
비율은 어느 위치에서
정보를 얼마나 가져올지
나타냅니다.

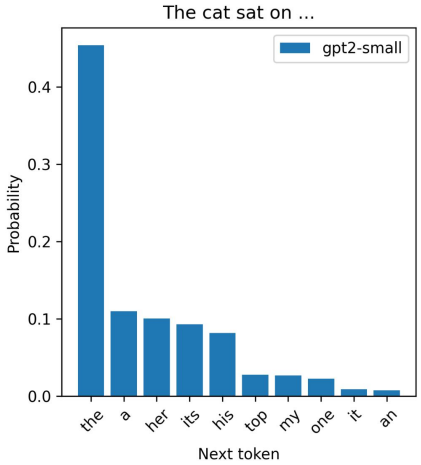
미래 위치의 참고 비율은
0이 되도록 막습니다.

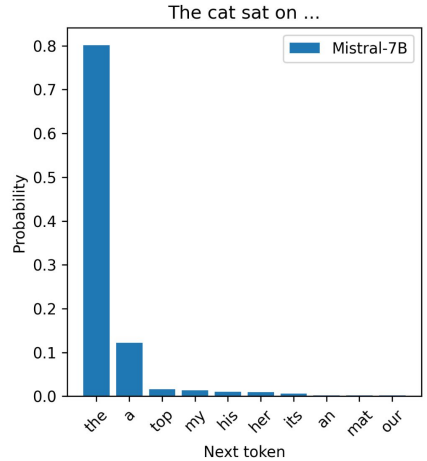
입력에서 서로 다른 변환으로
Q, K, V를 만듭니다.

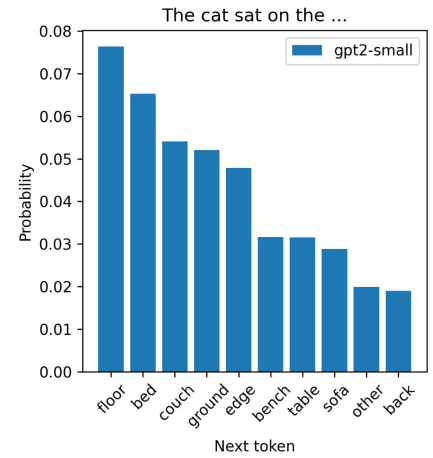


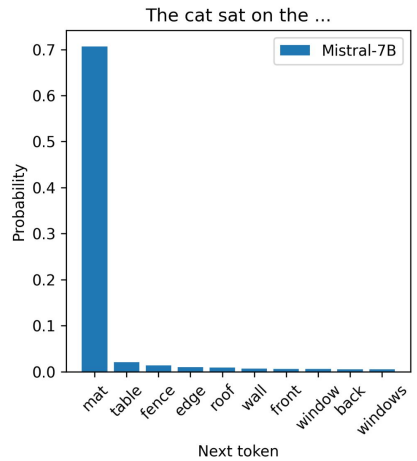
Q·K·V 만들기 → 점수 계산 → 미래 가리기 → 비율로 V 섞기 → 출력 변환











같은 모델도 앞에 주어진 말이 달라지면 다음 토큰의 확률이 달라집니다.

축 이름
b 문장 수 · s 토큰 수 · n 헤드 수
e 모델 벡터 길이 · h 헤드 벡터 길이

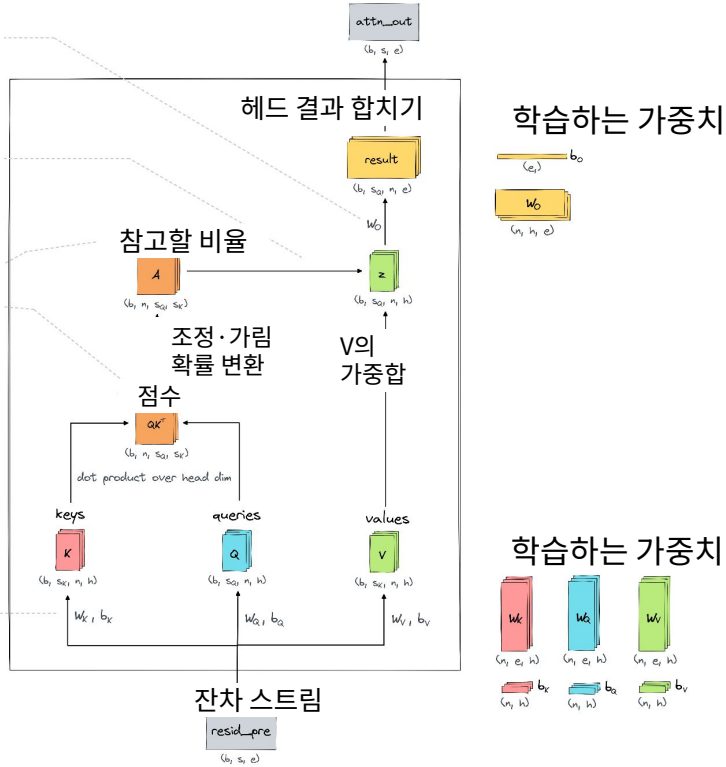
출력 변환 후
헤드 결과를 합칩니다.

V들을 참고할 비율에 따라
가중합합니다.

Q와 K를 내적해
점수를 계산합니다.

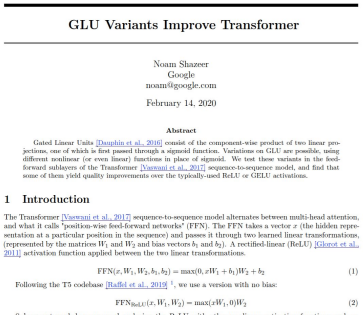
미래를 가린 뒤
소프트맥스를 적용합니다.

입력에서 Q, K, V를
각각 만듭니다.



고해상도 원본

각 화살표에서 숫자 배열의 크기를 확인하면, 어텐션 구현을 따라갈 수 있습니다.



GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

Abstract

Gated Linear Units [Dauphin et al., 2016] consist of the component-wise product of two linear projections, one of which is first passed through a sigmoid function. Variations on GLU are possible, using different nonlinear (or even linear) functions in place of sigmoid. We test these variants in the feed-forward sublayers of the Transformer [Vaswani et al., 2017] sequence-to-sequence model, and find that some of them yield quality improvements over the typically-used ReLU or GELU activations.

1 Introduction

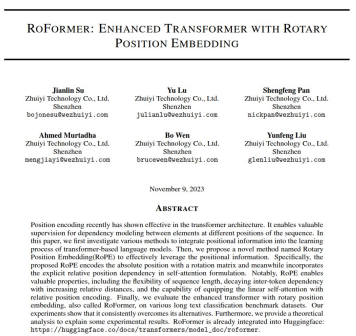
The Transformer [Vaswani et al., 2017] sequence-to-sequence model alternates between multi-head attention, and what it calls "position-wise feed-forward networks" (FFN). The FFN takes a vector x (the hidden representation at a particular position in the sequence) and passes it through two learned linear transformations, (represented by the matrices W_1 and W_2 and bias vectors b_1 and b_2). A rectified-linear (ReLU) [Glorot et al., 2011] activation function applied between the two linear transformations.

$$\mathrm{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1)$$

Following the T5 codebase [Raffel et al., 2019] [1], we use a version with no bias:

$$\mathrm{FFN}_{\mathrm{ReLU}}(x, W_1, W_2) = \max(xW_1, 0)W_2 \quad (2)$$

arXiv:2002.05202v1 [cs.LG] 12 Feb 2020



RoFormer: Enhanced Transformer with Rotary Position Embedding

Jianlin Su
Zhuiyi Technology Co., Ltd.
Shenzhen
bojonesu@wezhuiyi.com

Yu Lu
Zhuiyi Technology Co., Ltd.
Shenzhen
julianlu@wezhuiyi.com

Shengfeng Pan
Zhuiyi Technology Co., Ltd.
Shenzhen
nickpan@wezhuiyi.com

Ahmed Murtadha
Zhuiyi Technology Co., Ltd.
Shenzhen
mengjiayi@wezhuiyi.com

Bo Wen
Zhuiyi Technology Co., Ltd.
Shenzhen
brucewen@wezhuiyi.com

Yunfeng Liu
Zhuiyi Technology Co., Ltd.
Shenzhen
glenliu@wezhuiyi.com

November 9, 2023

Abstract

Position encoding recently has shown effective in the transformer architecture. It enables valuable supervision for dependency modeling between elements at different positions of the sequence. In this paper, we first investigate various methods to integrate positional information into the learning process of transformer-based language models. Then, we propose a novel method named Rotary Position Embedding(RoPE) to effectively leverage the positional information. Specifically, the proposed RoPE encodes the absolute position with a rotation matrix and meanwhile incorporates the explicit relative position dependency in self-attention formulation. Notably, RoPE enables valuable properties, including the flexibility of sequence length, decaying inter-token dependency with increasing relative distances, and the capability of equipping the linear self-attention with relative position encoding. Finally, we evaluate the enhanced transformer with rotary position embedding, also called RoFormer, on various long text classification benchmark datasets. Our experiments show that it consistently overcomes its alternatives. Furthermore, we provide a theoretical analysis to explain some experimental results. RoFormer is already integrated into Huggingface: https://huggingface.co/docs/transformers/model_doc/roformer.

Keywords Pre-trained Language Models · Position Information Encoding · Pre-training · Natural Language Processing.

1 Introduction

104.09864v5 [cs.CL] 8 Nov 2023



Swish: a Self-Gated Activation Function

Prajit Ramachandran; Barret Zoph, Quoc V. Le
Google Brain
{prajit,barretzoph,qvl}@google.com

Abstract

The choice of activation functions in deep networks has a significant effect on the training dynamics and task performance. Currently, the most successful and widely-used activation function is the Rectified Linear Unit (ReLU). Although various alternatives to ReLU have been proposed, none have managed to replace it due to inconsistent gains. In this work, we propose a new activation function, named Swish, which is simply $f(x) = x \cdot \mathrm{sigmoid}(x)$. Our experiments show that Swish tends to work better than ReLU on deeper models across a number of challenging datasets. For example, simply replacing ReLUs with Swish units improves top-1 classification accuracy on ImageNet by 0.9% for Mobile NASNet-A and 0.6% for Inception-ResNet-v2. The simplicity of Swish and its similarity to ReLU make it easy for practitioners to replace ReLUs with Swish units in any neural network.

1 Introduction

At the heart of every deep network lies a linear transformation followed by an activation function $f(\cdot)$. The activation function plays a major role in the success of training deep neural networks. Currently, the most successful and widely-used activation function is the Rectified Linear Unit (ReLU) (Hahnloser et al., 2000; Jarrett et al., 2009; Nair & Hinton, 2010), defined as $f(x) = \max(x, 0)$. The use of ReLUs was a breakthrough that enabled the fully supervised training of state-of-the-art deep networks (Krizhevsky et al., 2012). Deep networks with ReLUs are more easily optimized than networks with *sigmoid* or *tanh* units, because gradient is able to flow when the input to the ReLU function is positive. Thanks to its simplicity and effectiveness, ReLU has become the default activation function used across the deep learning community.

While numerous activation functions have been proposed to replace ReLU (Maas et al., 2013; He et al., 2015; Clevert et al., 2015; Klambauer et al., 2017), none have managed to gain the widespread adoption that ReLU enjoys. Many practitioners have favored the simplicity and reliability of ReLU because the performance improvements of the other activation functions tend to be inconsistent across different models and datasets.

We propose a new activation function that we name *Swish*:

$$f(x) = x \cdot \sigma(x) \quad (1)$$

arXiv:1710.05941v1 [cs.NE] 16 Oct 2017



Towards Understanding Mixture of Experts in Deep Learning

Zixiang Chen* and Yihe Deng† and Yue Wu‡ and Quanquan Gu§ and Yuanzhi Li¶

Abstract

The Mixture-of-Experts (MoE) layer, a sparsely-activated model controlled by a router, has achieved great success in deep learning. However, the understanding of such architecture remains elusive. In this paper, we formally study how the MoE layer improves the performance of neural network learning and why the mixture model will not collapse into a single model. Our empirical results suggest that the cluster structure of the underlying problem and the non-linearity of the expert are pivotal to the success of MoE. To further understand this, we consider a challenging classification problem with intrinsic cluster structures, which is hard to learn using a single expert. Yet with the MoE layer, by choosing the experts as two-layer nonlinear convolutional neural networks (CNNs), we show that the problem can be learned successfully. Furthermore, our theory shows that the router can learn the cluster-center features, which helps divide the input complex problem into simpler linear classification sub-problems that individual experts can conquer. To our knowledge, this is the first result towards formally understanding the mechanism of the MoE layer for deep learning.

Introduction

he Mixture-of-Expert (MoE) structure (Jacobs et al., 1991; Jordan and Jacobs, 1994) is a classic esign that substantially scales up the model capacity and only introduces small computation verhead. In recent years, the MoE layer (Eigen et al., 2013; Shazeer et al., 2017), which is an xtension of the MoE model to deep neural networks, has achieved remarkable success in deep arning. Generally speaking, an MoE layer contains many experts that share the same network rchitecture and are trained by the same algorithm, with a gating (or routing) function that routes ndividual inputs to a few experts among all the candidates. Through the sparse gating function, ne router in the MoE layer can route each input to the top-K ($K \geq 2$) best experts (Shazeer et al., 1 al., 2021). This routing scheme only costs the enjoys fast inference time.

California, Los Angeles, CA 90095, USA; e-mail:

California, Los Angeles, CA 90095, USA; e-mail:

alifornia, Los Angeles, CA 90095, USA; e-mail:

California, Los Angeles, CA 90095, USA; e-mail:

lon University, Pittsburgh, PA, USA; e-mail:

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023



Attentic

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam [illegible]*
Google Brain
noam@google.com

[illegible]*
Google Research
nikip@google.com

[illegible]*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

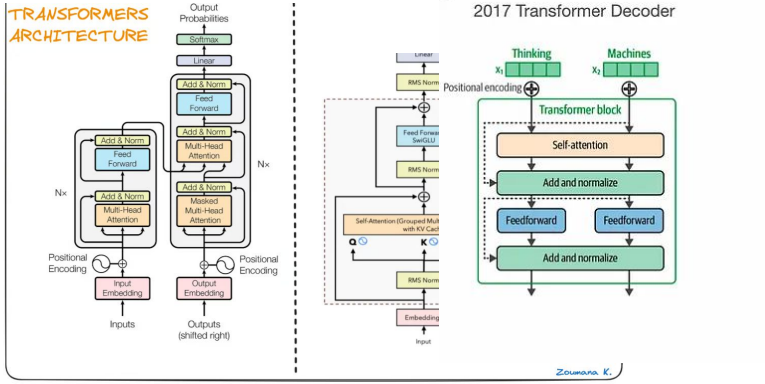
†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figur

구조를 바꾸면 성능이 달라질 수 있습니다.
그 이유가 언제나 완전히 밝혀진 것은 아닙니다.



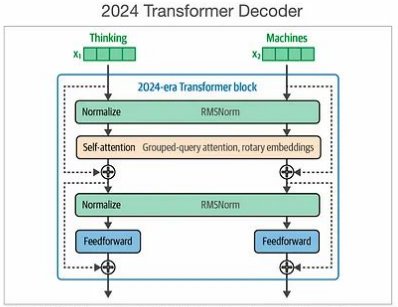


Figure 3-30. The Transformer block of a 2024-era Transformer like Llama 3 features some tweaks like pre-normalization and an attention optimized with grouped-query attention and rotary embeddings.

오늘은 GPT2의 기본 흐름을 이해하면 충분합니다. 구조의 변형은 다음 단계입니다.